

P2993R3

Extend `<bit>` header function with overloads for `std::simd`

Published Proposal, 2025-01-13

This version:

<http://wg21.link/P2933R3>

Authors:

[Daniel Towner](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to extend `std::simd` with overloads from other C++ standard libraries

Table of Contents

- 1 Motivation
- 2 Support for `<bit>`
- 3 Design confirmation request from LWG
- 4 Wording
 - 4.1 Add new section in [version.syn]
 - 4.2 Modify [`simd.expos`]

- 4.3 Update the synopsis
- 4.4 Add new section `[simd.bit]` after `[simd.math]`

5 Revision History

References

Informative References

§ 1. Motivation

[P1928R7] introduced data parallel types to C++. It mostly provided operators which worked on or with `std::simd` types, but it also included overloads of useful functions from other parts of C++ (e.g., `sin`, `cos`, `abs`). In this paper we propose some other functions from standard C++ headers which should receive overloads to work with `std::simd` types. The list isn't exhaustive, but reflects those functions which are desirable to include.

§ 2. Support for `<bit>`

The `<bit>` header is part of the numerics library and provides utilities for manipulating and querying the properties of integral values when treated as collections of bits. The table below summarises the contents of `<bit>`.

Name	Purpose	Proposed (Y/N)
<code>endian</code>	A type which indicates the endianness of scalar types.	N
<code>bit_cast</code>	reinterpret the object representation of one type as that of another	N
<code>byteswap</code>	reverses the bytes in the given integer value	Y
<code>has_single_bit</code>	checks if a number is an integral power of two	Y
<code>bit_ceil</code>	finds the smallest integral power of two not less than the given value	Y
<code>bit_floor</code>	finds the largest integral power of two not greater than the given value	Y
<code>bit_width</code>	finds the smallest number of bits needed to represent the given value	Y
<code>rotr</code>	computes the result of bitwise left-rotation	Y
<code>rotr</code>	computes the result of bitwise right-rotation	Y

<code>countl_zero</code>	counts the number of consecutive 0 bits, starting from the most significant bit	Y
<code>countl_one</code>	counts the number of consecutive 1 bits, starting from the most significant bit	Y
<code>countr_zero</code>	counts the number of consecutive 0 bits, starting from the least significant bit	Y
<code>countr_one</code>	counts the number of consecutive 1 bits, starting from the least significant bit	Y
<code>popcount</code>	counts the number of 1 bits in an unsigned integer	Y

Of these types and functions, only the first two shouldn't be handled by `std::simd`:

- `endian` indicates the endianness of a scalar type. A SIMD value with elements of the underlying scalar type will have the same properties and does not do a special handling in `std::simd`.
- `bit_cast` should be handled differently for `std::simd` values, and a separate proposal for `simd_bit_cast` will be provided.

All the other functions from `<bit>` should be handled in `std::simd` by element-wise application of the function to each element of the SIMD value. Any constraints and behaviours on the function will be applied at the SIMD value level. For instance, if `byteswap` participates in overload resolution only if the argument type satisfies `std::integral` concept then the overload of `byteswap` with `std::simd` parameter has the same constraint for `std::simd<T, N>::value_type`.

One small modification to the behaviour of `<bit>` for `std::simd` is where the return type differs to the input type. For example, the standard `<bit>` header defines some query functions as returning integer values:

```
template< class T >
constexpr int bit_width( T x ) noexcept;

template< class T >
constexpr int countl_one( T x ) noexcept;
```

If an `int` were to be returned from the `std::simd` overload of such functions then the size of the elements could change. For example, computing the bit width of a 8-bit integer could generate a `std::simd` of 64-bit integers as the output, which would lead to a dramatic change in storage size and performance. Instead, we propose that all the overloads for `<bit>` should return element types which are the same physical size as the element types they are querying. This would mean that calling `bit_width` on an `unsigned` 8-bit integer will return a `std::simd` containing signed 8-bit values.

When calling the rotate functions `rotl` and `rotr` functions it is common to want to rotate all simd elements by the same amount. An overload will be provided which takes a scalar `int` value to match `rotl` and `rotr` in the `<bit>` header. In this case there is no need to supply an integer of the same width as the first parameter's elements (as described above for the `simd` variant) since broadcasting an `integer` to a `simd` has negligible performance impact.

§ 3. Design confirmation request from LWG

During LWG review it was suggested that the `rotl` and `rotr` functions should provide an overload for the common case where a scalar could be used as the second parameter. This paper has been updated to reflect that change, and we would like LEWG to confirm acceptance of this design change.

§ 4. Wording

Below, substitute the  character with a number the editor finds appropriate for the table, paragraph, section or sub-section.

§ 4.1. Add new section in [version.syn]

In [version.syn] bump the `__cpp_lib_simd` version.

§ 4.2. Modify [simd.expos]

```
template<class V>
    concept simd-floating-point = // exposition only
        same_as<V, basic_simd<typename V::value_type, typename V::abi_type>>
        is_default_constructible_v<V> && floating_point<typename V::value_ty
```

```
template<class V>
    concept simd-type = // exposition only
        same_as<V, basic_simd<typename V::value_type, typename V::abi_type>>
        is_default_constructible_v<V>;
```

§ 4.3. Update the synopsis

In the header `<simd>` synopsis - [simd.syn] - add at the end after the "Mathematical functions"

```
// [simd.bit], Bit manipulation
template<simd-type V> constexpr V byteswap(const V& x) noexcept;
template<simd-type V> constexpr V bit_ceil(const V& x) noexcept;
template<simd-type V> constexpr V bit_floor(const V& x) noexcept;

template<simd-type V>
    constexpr typename V::mask_type has_single_bit(const V& x) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V rotl(const V& x, const V1& s) noexcept;
template<simd-type V0>
    constexpr V rotl(const V& x, int s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 rotr(const V0& x, const V1& s) noexcept;
template<simd-type V0>
    constexpr V0 rotr(const V0& x, int s) noexcept;

template<simd-type V>
    constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
        bit_width(const V& x) noexcept;
template<simd-type V>
    constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
        countl_zero(const V& x) noexcept;
template<simd-type V>
    constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
        countl_one(const V& x) noexcept;
template<simd-type V>
    constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
        countr_zero(const V& x) noexcept;
template<simd-type V>
    constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
        countr_one(const V& x) noexcept;
template<simd-type V>
    constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
        popcount(const V& x) noexcept;
```

§ 4.4. Add new section [**simd.bit**] after [**simd.math**]

◆ **basic_simd** bit library [**simd.bit**]

```
template<simd-type V> constexpr V byteswap(const V& x) noexcept
```

Constraints:

Any constraints from the corresponding scalar function `byteswap` are applied to `typename V::value_type`.

Returns:

The result of the element-wise application of `byteswap(x[i])` for all i in the range of $[0, V::size]$.

```
template<simd-type V> constexpr V bit_ceil(const V& x) noexcept;
template<simd-type V> constexpr V bit_floor(const V& x) noexcept;
template<simd-type V> constexpr typename V::mask_type has_single_bit(const
```

Constraints:

Any constraints from the corresponding scalar function from `<bit>` are applied to `typename V::value_type`.

Returns:

The result of the element-wise application of `bit-func(x[i])` for all i in the range of $[0, V::size]$, where `bit-func` is the corresponding scalar function from `<bit>`.

```
template<simd-type V0, simd-type V1> constexpr V0 rotl(const V0& v0, const
template<simd-type V0, simd-type V1> constexpr V0 rotr(const V0& v0, const
```

Constraints:

- Any constraints from the corresponding scalar function from `<bit>` are applied to `typename V0::value_type`.
- `sizeof(typename V0::value_type) == sizeof(typename V1::value_type)` is true.

Returns:

The result of the element-wise application of *bit-func*(*v0*[*i*], *v1*[*i*]) for all *i* in the range of [*0*, *V::size*], where *bit-func* is the corresponding scalar function from <bit>.

```
template<simd-type V> constexpr V rotl(const V& v, int s) noexcept;
template<simd-type V> constexpr V rotr(const V& v, int s) noexcept;
```

Constraints:

Any constraints from the corresponding scalar function from <bit> are applied to typename *V::value_type*.

Returns:

The result of the element-wise application of *bit-func*(*v*[*i*], *s*) for all *i* in the range of [*0*, *V::size*], where *bit-func* is the corresponding scalar function from <bit>.

```
template<simd-type V>
constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
bit_width(const V& v) noexcept;
template<simd-type V>
constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
countl_zero(const V& v) noexcept;
template<simd-type V>
constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
countl_one(const V& v) noexcept;
template<simd-type V>
constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
countr_zero(const V& v) noexcept;
template<simd-type V>
constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
countr_one(const V& v) noexcept;
template<simd-type V>
constexpr rebind_simd_t<std::make_signed_t<typename V::value_type>, V>
popcount(const V& v) noexcept;
```

Constraints:

Any constraints from the corresponding scalar function from <bit> are applied to typename *V::value_type*.

Returns:

The result of the element-wise application of *bit-func*(*v*[*i*], *s*) for all *i* in the range of `[0, V::size]`, where *bit-func* is the corresponding scalar function from `<bit>`. [Note: the type of the return value elements is the signed version of the input element type. This is different to the equivalent scalar functions which always return an *int* value.]

§ 5. Revision History

R2 => R3

- Add missing synopsis wording addition.
- Change detailed behaviour descriptions to match the style/wording of recent P1928 updates.
- Added concepts to detect simd types whose elements are integral, unsigned, and so on.
- Added overload to `rotr` to allow the second parameter to be a scalar.

R1 => R2

- Added feature test macro

R0 => R1

- Fix typo: `std::make_signed` to `std::make_signed_t`
- Make several paper text improvements

§ References

§ Informative References

[P1928R7]

Matthias Kretz. *[std::simd - Merge data-parallel types from the Parallelism TS 2](https://wg21.link/p1928r7)*. 15 October 2023. URL: <https://wg21.link/p1928r7>